OOP Interview

Questions & Answers Guide

Role: Software Development Engineer (SDE)

Topic: Object-Oriented Programming

Questions: 75+

Table of Contents

Section 1 — Core OOP Concepts

Q1 What are the four pillars of OOP?

Pillar	Definition	Real-World Analogy
Encapsulation	Bundling data + methods; hiding internal state	Medicine capsule — contents hidden inside
Abstraction	Exposing only necessary details; hiding complexity	Car dashboard — hides engine complexity
Inheritance	Child class acquires properties of parent	Son inheriting traits from father
Polymorphism	Same interface, different behaviour	'+' adds numbers AND concatenates strings

Q2 Difference between Abstraction and Encapsulation?

Aspect	Abstraction	Encapsulation
What it hides	Complexity / implementation	Data / internal state
How achieved	Abstract classes, Interfaces	Access modifiers (private/protected
Level	Design level	Implementation level
Goal	Show only what is needed	Protect data from outside access
Example	ATM shows 'Withdraw' button	ATM's PIN validation logic is private

■ ***Tip:*** *Abstraction is about WHAT an object does; Encapsulation is about HOW it is protected.*

Q3 What is Polymorphism? Explain its types with examples.

Compile-time (Static) Polymorphism — Method Overloading:

```
int    add(int a, int b)      { return a + b; }

double add(double a, double b) { return a + b; }

// Same name, different parameter types — resolved at compile time
```

Runtime (Dynamic) Polymorphism — Method Overriding:

```
class Animal { void sound() { System.out.println("..."); } }

class Dog extends Animal {

    @Override void sound() { System.out.println("Woof"); }

}

Animal a = new Dog();

a.sound();  // prints "Woof" — resolved at runtime via vtable
```

■ ***Tip:*** *Runtime polymorphism is achieved through method overriding + upcasting.*

Q4 Difference between Method Overloading and Overriding?

Aspect	Overloading	Overriding
Location	Same class	Parent-child classes
Signature	Must differ	Must be identical
Return type	Can differ	Must be same (or covariant)
Binding	Compile-time (static)	Runtime (dynamic)
Access modifier	Any	Cannot be more restrictive
static methods	Can be overloaded	Cannot be overridden (hidden)
private methods	Can be overloaded	Cannot be overridden

Q5 What is Inheritance? List all its types.

Type	Description	Supported in Java?
Single	A → B	Yes
Multilevel	A → B → C	Yes
Hierarchical	A → B and A → C	Yes
Multiple	A, B → C (two parents)	No (use Interface)
Hybrid	Combination of above	No (use Interface)

■ ***Tip:*** *Java disallows multiple class inheritance to avoid the Diamond Problem.*

Q6 What is the Diamond Problem and how is it solved?

Ambiguity arises when class D inherits from B and C, both of which override a method from A.

```
interface B { default void hello() { System.out.println("B"); } }

interface C { default void hello() { System.out.println("C"); } }

class D implements B, C {

    @Override

    public void hello() {

        B.super.hello();   // explicitly choose which parent's method to call

    }

}
```

■ ***Tip:*** *Java forces you to override and explicitly resolve the conflict.*

Q7 What is Encapsulation and why is it important?

Encapsulation means binding data (fields) and methods that operate on the data into a single unit (class), and restricting direct access to some of the object's components.

```
class BankAccount {
    private double balance;   // data hidden

    public void deposit(double amount) {
        if (amount > 0) balance += amount;  // controlled access
    }
    public double getBalance() { return balance; }
}
```

Benefits: Data protection | Modularity | Easier debugging | Loose coupling

Q8 What is Abstraction? How is it achieved in Java?

Abstraction hides complex implementation details and shows only the essential features. Achieved via:

- **Abstract classes** — partial abstraction (0–100%)
- **Interfaces** — full abstraction (100%)

```
abstract class Shape {
    abstract double area();    // abstract – no implementation
    void display() { System.out.println("Area: " + area()); }  // concrete
}
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
}
```

Q9 What is the difference between IS-A and HAS-A relationship?

Relationship	Meaning	Java Mechanism	Example
IS-A	Inheritance — child IS a type of parent	extends / implements	Dog IS-A Animal
HAS-A	Composition — class HAS another class as member	Instance variable	Car HAS-A Engine

■ ***Tip:*** *Prefer HAS-A (composition) over IS-A (inheritance) for flexibility — 'Favour composition over inheritance' is a GoF principle.*

Q10 What is Coupling and Cohesion?

	Coupling	Cohesion
Definition	Degree of dependency between classes	How focused a class is on a single responsibility
Goal	LOW coupling (less interdependency)	HIGH cohesion (single, well-defined job)
Bad example	Class A directly uses private fields of B	One class handles DB + UI + Business logic
Good example	A communicates with B only via interfaces	Separate classes for DB, UI, Business logic

Section 2 — Classes, Objects & Constructors

Q11 Difference between Class and Object?

Aspect	Class	Object
Definition	Blueprint / Template	Instance of a class
Memory	No memory allocated	Memory allocated at runtime
Created	Using 'class' keyword	Using 'new' keyword
Existence	Logical entity	Physical entity
Example	class Car { }	Car c = new Car();

Q12 What is a Constructor? List its types.

```
class Student {
    String name; int age;

    Student() { this.name = "Unknown"; this.age = 0; }         // Default
    Student(String n, int a) { this.name = n; this.age = a; }  // Parameterized
    Student(Student s) { this.name = s.name; this.age = s.age;}// Copy
}
```

Key facts: No return type | Same name as class | Called automatically on object creation

Q13 Can a constructor be private? When is it used?

Yes. Used in:

- **Singleton Pattern** — ensures only one instance
- **Factory Method Pattern** — controls object creation logic

```
class Singleton {
    private static Singleton instance;
    private Singleton() {}             // private — no external instantiation

    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

Q14 What is Constructor Chaining?

Calling one constructor from another within the same class (this()) or from parent (super()).

```
class Person {
    String name; int age;
    Person() { this("Unknown", 0); }         // calls parameterized constructor
    Person(String n, int a) { name=n; age=a; }
}

class Employee extends Person {
    String dept;
    Employee(String n, int a, String d) {
        super(n, a);                         // calls Person's constructor
        this.dept = d;
    }
}
```

■ ***Tip:*** *this() and super() must be the FIRST statement in a constructor.*

Q15 What is the difference between constructor and method?

Aspect	Constructor	Method
Name	Same as class name	Any valid identifier
Return type	No return type	Must have return type
Called by	JVM automatically on new	Explicitly by programmer
Inheritance	Not inherited	Inherited
Overriding	Cannot be overridden	Can be overridden

Q16 What is a Copy Constructor?

A constructor that creates a new object as a copy of an existing object.

```
class Box {
    int length, width;
    Box(Box b) {           // copy constructor
        this.length = b.length;
        this.width  = b.width;
    }
}
Box b1 = new Box(10, 5);
Box b2 = new Box(b1);    // creates a deep copy
```

■ ***Tip:*** *Java does not provide a default copy constructor unlike C++. You must write it manually.*

Q17 What is object cloning in Java?

Creating an exact field-by-field copy of an object using the Cloneable interface and clone() method.

- **Shallow Clone** — copies primitive values; object references are shared
- **Deep Clone** — copies everything including nested objects

```
class MyClass implements Cloneable {
    int x;
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();  // shallow copy
    }
}
```

Q18 What are instance, static, and local variables?

Type	Defined in	Memory	Default Value	Scope
Instance	Class (outside method)	Heap	Yes (0/null/false)	Object lifetime
Static	Class with static keyword	Method Area	Yes	Class lifetime
Local	Inside a method/block	Stack	No (must initialise)	Method/block scope

Q19 What is the 'this' keyword?

Refers to the current object instance. Uses:

- Disambiguate instance variable from local variable
- Call another constructor in the same class
- Pass current object as argument

```
class Person {
    String name;
    Person(String name) {
        this.name = name;       // disambiguate
    }
    Person() {
        this("Default");        // constructor chaining
    }
    void show(Printer p) {
        p.print(this);          // pass current object
    }
}
```

Q20 What is the 'super' keyword?

Refers to the immediate parent class. Uses:

- Access parent class methods/fields
- Call parent class constructor

```
class Animal {
    String type = "Animal";
    void sound() { System.out.println("Generic sound"); }
}
class Dog extends Animal {
    void sound() {
        super.sound();              // call parent method
        System.out.println("Woof");
    }
    Dog() { super(); }             // call parent constructor
}
```

Section 3 — Inheritance & Polymorphism

Q21 What is method hiding vs method overriding?

Aspect	Method Overriding	Method Hiding
Applies to	Instance methods	Static methods
Binding	Runtime (dynamic dispatch)	Compile-time (static)
Behaviour	Polymorphic — child's version runs	Based on reference type, not object type
Annotation	@Override recommended	@Override not applicable

```
class Parent {
    static void show() { System.out.println("Parent static"); }
    void display()     { System.out.println("Parent instance"); }
}
class Child extends Parent {
    static void show()  { System.out.println("Child static"); }   // HIDING
    @Override void display() { System.out.println("Child instance"); } // OVERRIDING
}
Parent p = new Child();
p.show();    // "Parent static" — hiding, resolved by reference type
p.display(); // "Child instance" — overriding, resolved by object type
```

Q22 Can we override private and static methods?

• **Private methods** — Cannot be overridden. They are not visible to subclasses, so the child class creates a new method, not an override.

• **Static methods** — Cannot be overridden. They belong to the class, not an instance. Child class can only hide them.

• **Final methods** — Cannot be overridden (compile-time error).

■ ***Tip:*** *Only non-static, non-private, non-final methods can be truly overridden.*

Q23 What is upcasting and downcasting?

Upcasting: Assigning child object to parent reference (implicit, always safe).

Downcasting: Casting parent reference back to child type (explicit, can throw ClassCastException).

```
Animal a = new Dog();   // upcasting — implicit

Dog d = (Dog) a;        // downcasting — explicit

// Safe downcasting using instanceof:

if (a instanceof Dog) {

    Dog dog = (Dog) a;

    dog.fetch();

}
```

Q24 What is Runtime Polymorphism? How does JVM achieve it?

Also called Dynamic Method Dispatch. When an overridden method is called through a parent reference, the JVM decides which method to invoke at runtime based on the actual object type, not the reference type.

JVM uses a **Virtual Method Table (vtable)** — a table of pointers to methods for each class. At runtime, the vtable of the actual object is consulted.

```
Shape s = new Circle();

s.area();   // JVM looks at Circle's vtable, not Shape's
```

Q25 What is an abstract method?

A method declared without a body (implementation). Forces subclasses to provide the implementation.

```
abstract class Vehicle {

    abstract void start();    // abstract — no body

    void stop() {             // concrete — has body

        System.out.println("Stopping...");

    }

}

class Car extends Vehicle {

    void start() { System.out.println("Car starting with key"); }

}
```

Rules: Must be in an abstract class | Cannot be private, static, or final

Q26 Can a subclass have different return type than parent?

Yes — called **Covariant Return Type**. The overriding method can return a subtype of the return type declared in the parent method.

```
class Animal {
    Animal create() { return new Animal(); }
}
class Dog extends Animal {
    @Override
    Dog create() { return new Dog(); }  // Dog is subtype of Animal — valid
}
```

■ ***Tip:*** *Introduced in Java 5. Only allowed for object types, not primitives.*

Q27 Can we call an overridden method from constructor?

Technically yes, but it is dangerous. If parent's constructor calls an overridden method, the child's version runs before the child's fields are initialised.

```
class Parent {
    Parent() { print(); }    // dangerous — calls overridden method
    void print() { System.out.println("Parent"); }
}
class Child extends Parent {
    int x = 10;
    @Override void print() { System.out.println("Child x = " + x); }
    // x is 0 here (not yet initialised) — prints "Child x = 0"
}
```

■ ***Tip:*** *Never call overridable methods from constructors — it can cause subtle bugs.*

Q28 What is the order of constructor execution in inheritance?

Top-down: Parent constructor always runs before child constructor.

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }

new C();
// Output:  A   B   C
```

Q29 Can we inherit a constructor?

No. Constructors are NOT inherited. A child class must define its own constructors. However, using super() you can explicitly call the parent's constructor.

■ ***Tip:*** *If you don't write super() explicitly, the compiler inserts super() as the first line, calling the parent's no-arg constructor.*

Q30 What is Object class in Java? Why is it important?

java.lang.Object is the root of the class hierarchy. Every class implicitly extends Object.

Important methods inherited from Object:

Method	Purpose
equals(Object o)	Logical equality check
hashCode()	Hash code for use in HashMap/HashSet
toString()	String representation of object
clone()	Create a copy of the object
getClass()	Return runtime class metadata
finalize()	Called before garbage collection (deprecated)
wait() / notify()	Thread synchronisation

Section 4 — Interfaces & Abstract Classes

Q31 Difference between Abstract Class and Interface?

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract; default/static (Java 8+)
Variables	Any type	public static final only
Constructor	Allowed	Not allowed
Multiple inheritance	Not supported	Supported
Access modifiers	Any	public by default
Speed	Faster	Slightly slower (extra indirection)
When to use	IS-A shared behaviour	CAN-DO capability contract

■ ***Tip:*** *Rule: IS-A relationship → Abstract class. CAN-DO capability → Interface.*

Q32 Can an abstract class have a constructor?

Yes. Although you cannot instantiate an abstract class directly, it can have a constructor that is called via super() from child class constructors.

```
abstract class Shape {
    String color;
    Shape(String color) { this.color = color; }   // constructor
}
class Circle extends Shape {
    double radius;
    Circle(String color, double r) {
        super(color);    // calls Shape's constructor
        this.radius = r;
    }
}
```

Q33 Can we instantiate an interface?

No directly. But you can via:

• **Anonymous class** implementing the interface

• **Lambda expression** (for functional interfaces)

```
interface Greeting { void greet(); }

// Anonymous class
Greeting g1 = new Greeting() {
    public void greet() { System.out.println("Hello!"); }
};

// Lambda (Java 8+) – only for functional interfaces (@FunctionalInterface)
Greeting g2 = () -> System.out.println("Hello!");
```

Q34 What are default and static methods in Interface (Java 8+)?

Default method: Provides a default implementation; can be overridden by implementing class.

Static method: Belongs to the interface itself; cannot be overridden.

```
interface Logger {
    default void log(String msg) {        // default method
        System.out.println("[LOG] " + msg);
    }
    static Logger getConsoleLogger() {    // static method
        return msg -> System.out.println(msg);
    }
}
```

■ ***Tip:*** *Default methods solve the 'interface evolution' problem — add new methods without breaking existing implementations.*

Q35 What is a Marker Interface?

An interface with no methods. It marks a class as having a certain property, used by JVM or frameworks.

Marker Interface	Purpose
Serializable	Marks class as serializable
Cloneable	Marks class as cloneable via clone()
RandomAccess	Marks list as O(1) random access

■ ***Tip:*** *Annotations like @Override, @Deprecated are modern replacements for marker interfaces.*

Q36 What is a Functional Interface?

An interface with exactly ONE abstract method. Used as the target type for lambda expressions.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);   // single abstract method
}

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;
System.out.println(add.calculate(3, 4));  // 7
```

Built-in functional interfaces: Runnable, Callable, Comparator, Predicate, Function, Supplier, Consumer

Q37 Can an interface extend another interface?

Yes. An interface can extend one or more interfaces.

```
interface A { void methodA(); }

interface B { void methodB(); }

interface C extends A, B { void methodC(); }   // multiple extension allowed

class D implements C {

    public void methodA() { }

    public void methodB() { }

    public void methodC() { }

}
```

■ ***Tip:*** *This is how multiple interface inheritance works — unlike classes, interfaces CAN extend multiple interfaces.*

Q38 When would you choose Abstract class over Interface and vice versa?

Choose Abstract Class when:

• You want to share code among closely related classes

• Classes have common fields/state

• You want to use access modifiers other than public

• Template method pattern is appropriate

Choose Interface when:

• Unrelated classes need to implement the same behaviour

• Multiple inheritance of type is needed

• You want to define a capability contract (Flyable, Serializable, etc.)

■ ***Tip:*** *In modern Java (8+), interfaces with default methods blur the line, but the design intent still matters.*

Section 5 — Access Modifiers & Keywords

Q39 Explain all four access modifiers in Java.

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default (no modifier)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Q40 What does the 'final' keyword do in Java?

Applied to	Effect	Example
Variable	Becomes constant — cannot be reassigned	final int MAX = 100;
Method	Cannot be overridden in subclass	final void display() {}
Class	Cannot be subclassed / extended	final class String {}

■ ***Tip:*** *String class is final — this is why strings are immutable in Java.*

Q41 What does the 'static' keyword mean?

Belongs to the class, not to any instance. Shared across all objects.

```
class Counter {
    static int count = 0;     // class variable – shared
    int id;                   // instance variable – unique per object
    Counter() { id = ++count; }
}
// All Counter objects share the same 'count'
```

static method — can call only static members; no access to 'this'

static block — runs once when class is loaded

static inner class — does not need outer class instance

Q42 What is the difference between '==' and equals()?

Aspect	==	equals()
Compares	References (memory address)	Content / logical equality
For primitives	Compares values	N/A (no method on primitives)
Default in Object	Same as ==	Same as == (unless overridden)
String behavior	Checks same object	Checks same character sequence

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a == b);        // false — different objects
System.out.println(a.equals(b));   // true  — same content
```

Q43 Why override hashCode() when overriding equals()?

Contract: Objects that are equal (equals() returns true) MUST have the same hashCode(). If only equals() is overridden, HashMap/HashSet will break because they use hashCode() first to find the bucket.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Person)) return false;
    return this.id == ((Person) o).id;
}
@Override
public int hashCode() {
    return Objects.hash(id);   // must be consistent with equals
}
```

Q44 What is the 'instanceof' operator?

Tests whether an object is an instance of a specific class or interface. Returns boolean.

```
Animal a = new Dog();
System.out.println(a instanceof Animal);  // true
System.out.println(a instanceof Dog);     // true
System.out.println(a instanceof Cat);     // false

// Java 16+ pattern matching:
if (a instanceof Dog d) {
    d.fetch();   // 'd' is already downcast
}
```

Q45 What is the volatile keyword?

Ensures a variable is read from and written to main memory directly, not from CPU cache. Used in multithreading to ensure visibility of changes across threads.

```
class SharedFlag {
    volatile boolean running = true;  // visible to all threads

    void stop() { running = false; }
    void run() {
        while (running) {  // always reads fresh value
            // do work
        }
    }
}
```

■ ***Tip:*** *volatile ensures visibility but NOT atomicity. Use AtomicInteger or synchronized for compound operations.*

Q46 What is the transient keyword?

Marks a field to be excluded from serialization. When the object is serialized, transient fields are skipped and set to their default values upon deserialization.

```
class User implements Serializable {

    String username;

    transient String password;   // excluded from serialization

}
```

Section 6 — Advanced OOP Concepts

Q47 What is Composition vs Inheritance? Which is preferred?

Inheritance creates an IS-A relationship (tight coupling). **Composition** creates a HAS-A relationship (loose coupling).

```
// Inheritance — tightly coupled
class Logger extends File { ... }  // Logger IS-A File? No — bad design

// Composition — loosely coupled (preferred)
class Logger {
    private File logFile;           // Logger HAS-A File
    Logger(File f) { logFile = f; }
}
```

■ ***Tip:*** *GoF Principle: Favour composition over inheritance. Composition is more flexible — you can change the component at runtime.*

Q48 What is Association, Aggregation, and Composition?

Relationship	Strength	Lifecycle	Example
Association	Weakest	Independent	Teacher uses Projector
Aggregation	Weak (HAS-A)	Independent	Department HAS Professors
Composition	Strong (PART-OF)	Dependent	House PART-OF Rooms

- **Composition:** If House is destroyed, Rooms are destroyed too
- **Aggregation:** If Department is closed, Professors still exist

Q49 What is an inner class? List its types.

Type	Description	Can access outer?
Member inner class	Non-static class inside a class	Yes — all members
Static nested class	Static class inside a class	Only static members
Local inner class	Class inside a method	Yes + method's final locals
Anonymous inner class	Nameless one-time use class	Yes + final locals

Q50 What is object-oriented design (OOD)? Key principles?

OOD is the process of designing a software system using OOP concepts. Key design goals:

- **Modularity** — break system into independent modules
- **Extensibility** — easy to add new features without changing existing code
- **Reusability** — components can be reused across projects
- **Maintainability** — easy to change, test, and debug

■ ***Tip:*** *Good OOD = Low coupling + High cohesion + SOLID principles.*

Q51 What is the difference between early binding and late binding?

Aspect	Early Binding (Static)	Late Binding (Dynamic)
When resolved	Compile time	Runtime
Applies to	Overloaded methods, static methods	Overridden instance methods
Performance	Faster	Slightly slower (vtable lookup)
Flexibility	Less flexible	More flexible (polymorphism)

Q52 What is the Liskov Substitution Principle (LSP)?

Objects of a superclass should be replaceable with objects of a subclass without altering the correctness of the program.

```
// LSP VIOLATION:

class Rectangle { int w, h; setWidth(int w) { this.w = w; } setHeight(int h) { this.h
 = h; } }

class Square extends Rectangle {

    @Override setWidth(int w) { this.w = this.h = w; }   // breaks Rectangle's behavi
our

}


// LSP FIX: make both inherit from a common Shape interface instead
```

■ ***Tip:*** *If a child class overrides a parent method in a way that breaks expected behaviour, it violates LSP.*

Q53 Explain method resolution order (MRO) in multiple interface inheritance.

When a class implements multiple interfaces with the same default method, Java forces explicit override to resolve ambiguity.

```
interface A { default void greet() { System.out.println("A"); } }
interface B { default void greet() { System.out.println("B"); } }

class C implements A, B {
    @Override
    public void greet() {
        A.super.greet();  // explicitly choose — required or compile error
    }
}
```

Q54 What is the difference between Shallow Copy and Deep Copy?

Aspect	Shallow Copy	Deep Copy
Primitives	Copied	Copied
Object references	Reference copied (shared)	New object created
Independence	Not fully independent	Fully independent
How	clone() default / copy constructor	Manual / serialization / libraries

```
// Deep copy example
class Address { String city; }
class Person {
    String name; Address addr;
    Person deepCopy() {
        Address newAddr = new Address();
        newAddr.city = this.addr.city;   // copy nested object
        Person p = new Person();
        p.name = this.name;
        p.addr = newAddr;
        return p;
    }
}
```

Q55 What is Immutability? How to create an immutable class?

An immutable object's state cannot be changed after creation. Examples: String, Integer, LocalDate.

Steps to create immutable class:

1. Declare class as final
2. Make all fields private and final
3. No setter methods
4. Deep copy mutable objects in constructor and getters

```
public final class ImmutablePoint {
    private final int x, y;
    public ImmutablePoint(int x, int y) { this.x = x; this.y = y; }
    public int getX() { return x; }
    public int getY() { return y; }
    // no setters
}
```

Q56 What is method chaining? How is it implemented?

Method chaining allows calling multiple methods on the same object sequentially. Implemented by returning 'this' from each method.

```
class Builder {
    private String name; private int age;
    Builder setName(String name) { this.name = name; return this; }
    Builder setAge(int age)      { this.age  = age;  return this; }
    Person build() { return new Person(name, age); }
}

Person p = new Builder().setName("Alice").setAge(30).build();
```

■ ***Tip:*** *This is the foundation of the Builder design pattern.*

Q57 What is the difference between Procedural and OOP?

Aspect	Procedural	OOP
Focus	Functions / procedures	Objects and classes
Data	Exposed globally or passed around	Encapsulated inside objects
Reusability	Limited (function reuse)	High (inheritance + composition)
Approach	Top-down	Bottom-up
Examples	C, Pascal	Java, C++, Python, C#

Section 7 — Design Principles & Patterns

Q58 Explain SOLID Principles.

Letter	Principle	Meaning	Violation Example
S	Single Responsibility	One class = one reason to change	UserManager handles DB + Email + PDF
O	Open/Closed	Open for extension, closed for modification	Adding if-else for every new shape
L	Liskov Substitution	Subtypes must be substitutable for base types	Square breaks Rectangle contract
I	Interface Segregation	Don't force unused methods on a class	Fat interface with 20 methods
D	Dependency Inversion	Depend on abstractions, not concretions	High-level module imports MySQL directly

Q59 What is the Singleton Pattern? Implement a thread-safe version.

```
public class Singleton {
    // volatile ensures visibility; double-checked locking for performance
    private static volatile Singleton instance;
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
// Alternative: Bill Pugh (best approach — lazy, thread-safe, no sync overhead)
class BillPugh {
    private BillPugh() {}
    private static class Holder { static final BillPugh INSTANCE = new BillPugh(); }
    public static BillPugh get() { return Holder.INSTANCE; }
}
```

Q60 What is the Factory Pattern?

Defines an interface for creating an object, but lets subclasses decide which class to instantiate. Promotes loose coupling.

```
interface Animal { void speak(); }

class Dog implements Animal { public void speak() { System.out.println("Woof"); } }

class Cat implements Animal { public void speak() { System.out.println("Meow"); } }

class AnimalFactory {

    static Animal create(String type) {

        return switch (type) {

            case "dog" -> new Dog();

            case "cat" -> new Cat();

            default    -> throw new IllegalArgumentException("Unknown: " + type);

        };

    }

}

Animal a = AnimalFactory.create("dog");  // client doesn't know about Dog
```

Q61 What is the Observer Pattern?

Defines a one-to-many dependency. When one object (subject) changes state, all dependents (observers) are notified automatically.

Real-world: Event listeners, MVC (Model notifies Views), stock price feeds, social media notifications.

```
interface Observer { void update(String event); }

class EventBus {

    List<Observer> listeners = new ArrayList<>();

    void subscribe(Observer o) { listeners.add(o); }

    void publish(String event) {

        for (Observer o : listeners) o.update(event);

    }

}
```

Q62 What is the Strategy Pattern?

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Lets the algorithm vary independently from clients that use it.

```
interface SortStrategy { void sort(int[] arr); }

class BubbleSort implements SortStrategy { public void sort(int[] a) { /* ... */ } }

class QuickSort  implements SortStrategy { public void sort(int[] a) { /* ... */ } }

class Sorter {

    private SortStrategy strategy;

    Sorter(SortStrategy s) { this.strategy = s; }

    void setStrategy(SortStrategy s) { this.strategy = s; }

    void sort(int[] arr) { strategy.sort(arr); }

}
```

■ ***Tip:*** *Strategy = OCP in action. Add new algorithms without changing the client.*

Q63 What is the Decorator Pattern?

Attaches additional responsibilities to an object dynamically. Provides a flexible alternative to subclassing for extending functionality.

```
interface Coffee { double cost(); }

class SimpleCoffee implements Coffee { public double cost() { return 1.0; } }

class MilkDecorator implements Coffee {

    private Coffee coffee;

    MilkDecorator(Coffee c) { this.coffee = c; }

    public double cost() { return coffee.cost() + 0.5; }

}

Coffee c = new MilkDecorator(new SimpleCoffee());  // base + milk

// Can stack: new SugarDecorator(new MilkDecorator(new SimpleCoffee()))
```

Q64 What is Dependency Injection (DI)?

A technique where an object's dependencies are provided from outside rather than created internally. Promotes loose coupling and testability.

Type	How	Example
Constructor Injection	Via constructor	new Service(repo)
Setter Injection	Via setter method	service.setRepo(repo)
Field Injection	Via @Autowired (Spring)	@Autowired Repository repo

```
// Without DI (tight coupling)
class OrderService { private PayPal payment = new PayPal(); }

// With DI (loose coupling)
class OrderService {
    private PaymentGateway payment;
    OrderService(PaymentGateway p) { this.payment = p; }  // inject dependency
}
```

Q65 What is the Template Method Pattern?

Defines the skeleton of an algorithm in a base class, deferring some steps to subclasses. Subclasses can override certain steps without changing the algorithm's structure.

```
abstract class DataProcessor {
    // Template method — defines the skeleton
    final void process() {
        readData();
        processData();   // abstract — subclass implements
        writeData();
    }
    abstract void processData();
    void readData()  { System.out.println("Reading..."); }
    void writeData() { System.out.println("Writing..."); }
}
class CSVProcessor extends DataProcessor {
    void processData() { System.out.println("Processing CSV..."); }
}
```

Section 8 — Tricky / Viva Questions

Q66 Is Java 100% object-oriented?

No. Java is NOT 100% object-oriented because:

• It has 8 primitive types (int, char, boolean, etc.) that are not objects

• Static methods/variables belong to the class, not an object

• Wrapper classes (Integer, Boolean) partially solve this via autoboxing

■ ***Tip:*** *Languages like Smalltalk and Ruby are 100% OOP — everything is an object.*

Q67 Can we override the main() method?

No. main() is a static method, and static methods cannot be overridden — they can only be hidden. Also, the JVM specifically looks for the signature public static void main(String[] args). A different signature is simply treated as a new method.

Q68 What happens if we don't override equals() and hashCode()?

• equals() falls back to Object's default which uses == (reference comparison)

• Two logically equal objects will not be recognised as equal

• HashSet/HashMap will store duplicates since hashCode() returns different values

```
Set<Person> set = new HashSet<>();

set.add(new Person("Alice", 1));

set.add(new Person("Alice", 1));  // duplicate not detected without override

System.out.println(set.size());   // 2 instead of 1
```

Q69 Can an abstract class implement an interface?

Yes. An abstract class can implement an interface without providing implementations for all interface methods. The concrete subclass must then implement the remaining methods.

```
interface Flyable { void fly(); void land(); }

abstract class Bird implements Flyable {

    public void land() { System.out.println("Landing..."); }

    // fly() left abstract — subclass must implement

}

class Eagle extends Bird {

    public void fly() { System.out.println("Eagle flying"); }

}
```

Q70 What is the difference between String, StringBuilder, and StringBuffer?

Aspect	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safety	Yes (immutable)	No	Yes (synchronised)
Performance	Slow (new object each change)	Fast	Slower than StringBuilder
Use when	Fixed strings	Single-thread string ops	Multi-thread string ops

Q71 Can we have a try block without a catch block?

Yes, if it has a finally block. Valid forms:

• try { } catch { }

• try { } finally { }

• try { } catch { } finally { }

• try-with-resources (auto-closes resources implementing AutoCloseable)

```
try (FileReader fr = new FileReader("file.txt")) {

    // auto-closed after block

} catch (IOException e) {

    e.printStackTrace();

}
```

Q72 What is the difference between checked and unchecked exceptions?

Aspect	Checked Exception	Unchecked Exception
Checked at	Compile time	Runtime
Must handle	Yes (try-catch or throws)	No
Extends	Exception	RuntimeException
Examples	IOException, SQLException	NullPointerException, ArrayIndexOutOfBoundsException
Cause	External factors (file not found)	Programming errors (null dereference)

Q73 What is the difference between throw and throws?

Aspect	throw	throws
Purpose	Actually throws an exception	Declares exceptions a method may throw
Location	Inside method body	In method signature
Followed by	Exception instance	Exception class names
Multiple	Only one at a time	Can list multiple

```
void validate(int age) throws IllegalArgumentException {
    if (age < 0) throw new IllegalArgumentException("Age cannot be negative");
}
```

Q74 What is Garbage Collection in Java?

Automatic memory management — JVM automatically reclaims memory occupied by objects no longer reachable.

GC Algorithms: Serial GC | Parallel GC | G1 GC (default, Java 9+) | ZGC (low-latency)

GC Roots: Stack variables, static fields, JNI references

finalize(): Called before GC removes object — deprecated in Java 9 (use try-with-resources or Cleaner instead)

■ ***Tip:*** *You cannot force GC but can request it via System.gc() — not guaranteed to run.*

Q75 Quick Fire Round — 10 Must-Know Answers

#	Question	Answer
1	Can interfaces have constructors?	No
2	Can abstract class be final?	No — final prevents extension; abstract requires it
3	Can static method access instance var?	No — belongs to class, no 'this'
4	Multiple inheritance possible in Java?	Via interfaces only, not classes
5	Can we instantiate abstract class?	No (only via anonymous class)
6	What is object-level locking?	synchronized on instance method
7	What is class-level locking?	synchronized on static method
8	toString() default output?	ClassName@hexHashCode
9	Can constructor return a value?	No — it has no return type
10	Is String a primitive in Java?	No — it is a class (reference type)

Best of Luck for Your SDE Interview!

Covers 75 questions across 8 sections — Core OOP | Classes | Inheritance | Interfaces | Keywords | Advanced | Design Patterns | Tricky Viva